



Prompt Ensembling

Prompt Ensembling

1) What it is

Prompt ensembling runs the **same task** through **multiple prompt formulations** (often with identical model/params) and **aggregates** the results (vote/rank/score/select). It reduces variance and improves reliability by leveraging **diverse prompt perspectives**.

Contrast

- *Self-consistency*: multiple **samples from one prompt** (vary temperature).
 - *Prompt ensembling*: multiple **prompts** (wording/structure/examples), usually sampled deterministically.
-

2) When to use

- **Critical correctness** tasks (classification, extraction, judgment).
- **Style/format sensitivity** (schemas, safety wording).
- **Ambiguous inputs** where phrasing affects interpretation.
- **Evaluation** (LLM-as-judge): mitigate bias of a single rubric.

Avoid when latency/cost must be minimal and a single well-tuned prompt already meets SLAs.

3) Ensemble design patterns

3.1 Diversity sources (how prompts differ)

- **Instruction framing**: bullet rules vs. numbered SOP; active vs. passive voice.

- **Role framing:** "auditor," "teacher," "CSR," etc.
- **Order & emphasis:** constraints-first vs. examples-first.
- **Few-shot choices:** different exemplars / negative examples.
- **Structure contract:** strict JSON vs. Markdown sections.
- **Decoding knobs:** keep similar (low-temp); only vary if intentionally exploring.

3.2 What you ensemble

- **Outputs** (final answers, JSON objects).
- **Scores** (each prompt returns a confidence/rationale you can combine).
- **Rationales** (use a judge to rate them, then select the best final answer).

4) Aggregation strategies (how to combine)

Method	Works Best For	How
Majority vote	Single-label classification	Choose label with most votes; tie → fallbacks
Rank aggregation	Top-k lists, search results	Borda/Reciprocal Rank; merge deduped items
Median/trimmed mean	Numeric outputs	Remove extremes; average rest
Consensus JSON	Structured extraction	Field-wise vote/merge; require schema validity
LLM-as-Judge	Open-ended text	Judge compares candidates using a rubric; pick winner
Meta-scoring	Mixed	Score by (format adherence × rubric × self-confidence); pick max

Tie-breakers: prefer outputs that (1) **validate** against schema, (2) **cite sources** or supply **evidence**, (3) **meet constraints** (length, sections).

5) Minimal controller workflow

1. **Generate variants:** 3–7 concise prompts with different framings.
2. **Run:** call model once per prompt (low temperature, e.g., 0–0.3).

3. **Validate**: schema/regex/length/safety checks.
 4. **Aggregate**: majority vote / rank merge / judge selection.
 5. **Fallback**: if no candidate passes validation → repair pass (ask for corrected JSON) or return “insufficient data.”
 6. **Log**: keep per-prompt scores and decisions (for future pruning).
-

6) Example (structured extraction)

Task: Extract `{company, country, foundedYear}` from a paragraph.

Prompts P1–P3: vary wording (rules-first vs. examples-first), and few-shot pairs.

Aggregation:

- Validate JSON → discard invalid.
- For each field: choose value with **majority**; if all differ → pick value with **most supporting evidence** (citations or quote spans).

Judge rubric (copy-paste)

```
Compare Candidate A vs B for factual extraction.
Score 1–5 on: (1) correctness vs source text, (2) schema ad
herence, (3) completeness.
Return JSON: {"winner": "A|B|tie", "reason": "...", "score
A": n, "scoreB": n}
```

7) Example (classification + majority vote)

```
Labels: {PlaceOrder, ModifyOrder, CancelOrder, Status, Othe
r}
```

```
Run prompts: P_role, P_rulesFirst, P_examplesFirst (temp=0.
2)
```

Aggregate:

- Take majority label.
- If tie: prefer label predicted by a prompt with higher pa

```
st F1 on dev set.
```

- Log confusion pairs for future prompt pruning.

8) Cost, latency, and quality tips

- **Small, strong ensemble:** 3–5 diverse prompts beat 10 similar ones.
- **Parallelize** calls; **short prompts** with strict formats reduce tokens.
- **Cache** per-prompt outputs; reuse across retries.
- **Champion/Challenger:** ship a 3-prompt ensemble; trial additional prompts on a traffic slice.
- **Prune** prompts that rarely win or violate schemas.

9) Pitfalls & how to avoid them

Pitfall	Fix
Prompts are too similar → low diversity	Enforce orthogonal differences (role, order, examples)
Cost/latency blow-up	Cap ensemble to 3–5; parallelize; shorten prompts; use cheaper model for prefilter
Judge bias to verbosity	Judge against a rubric ; cap length; normalize formats
Conflicting schemas	Use one canonical schema; auto-map fields from each prompt
Overfitting to dev set	Keep a hold-out test; rotate inputs; track drift

10) Controller pseudocode (copy-paste)

```
prompts = [P_rules_first, P_examples_first, P_role_auditor]
# diverse!
candidates = [call_llm(p, x, temperature=0.2) for p in prompts]

valid = [c for c in candidates if schema_valid(c)]
if not valid:
    repaired = [repair_json(c) for c in candidates]
```

```

    valid = [v for v in repaired if schema_valid(v)]

    if task == "classification":
        label = majority_vote([v["label"] for v in valid])
        if tie(label):
            label = fallback_by_historic_f1(valid, dev_stats)
        return label

    if task == "generation":
        winner = llm_judge_compare(valid, rubric=rubric_text)
        return winner

```

11) Ready-to-use prompt skeletons

P_rulesFirst

System: You are a precise, policy-bound extractor.
 Rules: Return ONLY valid JSON; no inference—if missing, use null.
 Schema: { ... }
 Task: Extract fields from the text.
 Text: <<<DOC>>>

P_examplesFirst

Follow the schema below. See examples; mirror the structure exactly.

Example:

Input: "Acme, founded in 2012 in Canada..."

Output: {"company": "Acme", "country": "Canada", "foundedYear": 2012}

Now extract from:

<<<DOC>>>

Return ONLY JSON.

P_roleAuditor

Act as a compliance auditor. Use exact spans from the text as evidence.

Return:

```
{ "company": "...", "country": "...", "foundedYear": 2012,
  "evidence": {"company":"...", "country":"...", "foundedYear":"..."} }
```

12) Evaluation & monitoring

- **Offline:** macro-F1 / EM (classification), **schema validity %**, **adherence** (length/sections), **judge win-rate**.
- **Online:** CSAT, escalation rate, latency, token cost, safety flags.
- **Drift:** track per-prompt win shares; retire weak prompts; add new challengers quarterly.

13) Checklist (copy-paste)

- ☐ 3–5 **diverse** prompts (role, order, examples, structure)
- ☐ Low temperature for determinism
- ☐ Single canonical schema + validator
- ☐ Aggregation rule (vote/rank/judge) + tie-breakers
- ☐ Parallel execution + caching
- ☐ Metrics dashboard (accuracy/adherence/cost/latency)
- ☐ Champion/Challenger rotation & pruning schedule

Key Takeaway

Prompt ensembling is **variance reduction for prompts**: combine a few **meaningfully different** prompts, **validate** their outputs, and **aggregate** with simple rules (vote/rank/judge). Done right, it delivers **steadier accuracy** and **production-grade reliability** with manageable cost.